

Estimation-Pruning (EP) Algorithm for Point-to-Point Travel Cost Minimization in a Non-FIFO Dynamic Network

Keyvan Mohajer, Almir Mutapcic, and Majid Emami

Abstract. This paper presents the estimation-pruning (EP) algorithm for finding the best path (with minimum cost) from a source to a destination in a dynamic network that does not necessarily obey the first-in-first-out (FIFO) property. The EP algorithm consists of two steps. The first step is the forward or the estimation step in which a bound on the traveling cost of each possible path is calculated. The second step is the backward or the pruning step in which the paths that are unlikely to produce the best route are eliminated. The resulting network is then expanded in time and is converted to a static network, which is used to find the best route.

Index Terms: dynamic network routing, estimation-pruning algorithm, Dijkstra algorithm, network optimization, FIFO property.

I. Introduction

THE problem of finding the best route from a source to a destination in a weighted network has received a great deal of attention in the past few decades [1], [2]. This problem arises in many contexts; for example, data packets traveling through a computer network and vehicles traveling on a road network would ideally choose the optimum path according to some predefined criteria. For the vehicle routing problem, commercially available tools already exist that allow a user to specify a source and a destination and receive the corresponding route [3]–[5]. These tools mainly focus on optimizing the travel time of the route. In some instances, however, the user may desire to optimize the travel cost based on other factors such as safety, distance, standard deviation, avoidance of speed traps, and other considerations. In fact the cost of each link in the network can be computed according to a weighted combination of the above factors specified by the user. The problem gets more complicated once we start dealing with a dynamic network (a network in which the link travel costs are time dependent [6]–[8]). Moreover, when the defined travel cost does not have the FIFO property (defined below), the situation gets significantly more difficult. This paper presents a novel algorithm (The Estimation-Pruning Algorithm) to find a cluster of routes in the network that have the best bound on their travel cost. This algorithm has very good running time characteristics.

The next section in this paper describes the nature of non-FIFO dynamic networks in detail and introduces the idea of time expansion of a dynamic network. The basic ideas of the estimation and the pruning steps in the EP algorithm are explained in sections III and IV respectively. Section V presents the EP algorithm itself. The running time analysis is described in section VI and section VII provides an extension to the basic EP algorithm of section V for more accurate results at the expense of longer processing time using a tuning parameter. Finally the simulation results are presented in section VIII.

II. Non-FIFO Dynamic Networks and Time Expansion

Let $G = (V, E)$ be a directed network, where $V = 1, 2, \dots, N$ is the set of nodes and $E = 1, 2, \dots, M$ is the set of directed links. The cost of traveling from node i to node j in a dynamic network is a function of time and is represented as $C_{ij}(t)$. We will assume that $C_{ij}(t)$ is a discrete function of time and stays constant for periods of Δt . Furthermore, we will assume that there is at most one link from node i to node j , although this is not required and is only assumed for simplicity of this presentation. In addition, the time it takes to travel from node i to node j in a dynamic network is also a function of time and is represented by $T_{ij}(t)$. The parameter $T_{ij}(t)$ is not necessarily the same as $C_{ij}(t)$ since the traveling cost could be a function of traveling time and other factors.

Consider the simple example of Fig. 1. Assume that the user can leave node A between times t_1 and $t_2 = t_1 + 3\Delta t$ and would like to reach node C with the minimum total traveling cost.

Depending on when the user departs from node A , the cost of going from node A to node B could be $C_{AB}(t_1)$, $C_{AB}(t_1 + \Delta t)$, $C_{AB}(t_1 + 2\Delta t)$ or $C_{AB}(t_1 + 3\Delta t)$. Similarly, the cost of going from node A to node C depends on when the user departs from node A . The cost of going from node B to node C depends on when the user arrives at node B . For instance, If the user leaves node A at time $t_1 + \Delta t$, she will get to node B at time $t_1 + \Delta t + T_{AB}(t_1 + \Delta t)$ and the cost of going from B to C would be $C_{BC}(t_1 + \Delta t + T_{AB}(t_1 + \Delta t))$.

In the process of finding the best route with minimum cost, one should consider all possibilities. To do this, one can expand the dynamic network in time by replicating each node to convert it to a static network. Fig. 2 shows the corresponding time expanded network of Fig. 1.

In this case, node A has been replicated four times to account for the fact that the user can leave during four different time intervals. Note that in each case, the user can arrive at node C in two different time slots (depending on which path is chosen). Therefore, if the user wants to leave node C , the time-expanded version of Fig. 1 would have $4 \times 2 = 8$ different replications. The time expansion technique described in [9] increases the size of the

The authors are Ph.D. candidates with the Department of Electrical Engineering, Stanford University, Stanford, CA. Email: {keyvan, almir, memami}@stanford.edu

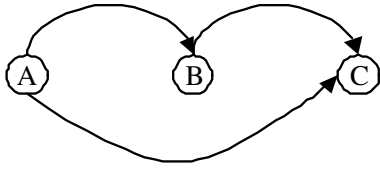


Fig. 1. Sample Network

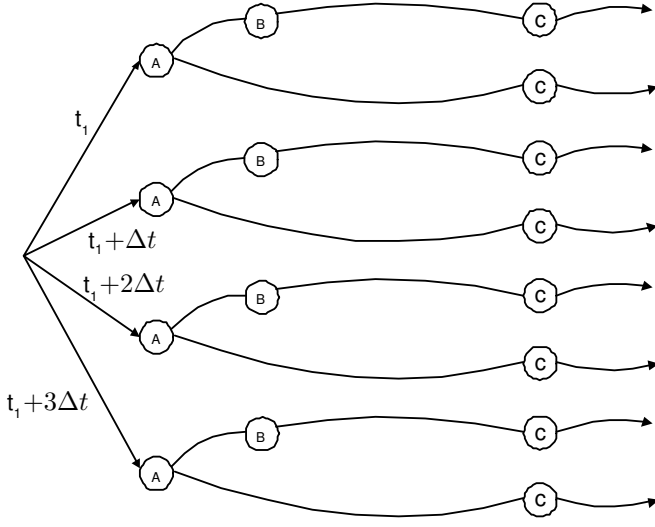


Fig. 2. Time expanded network

dynamic network by the number of available time intervals. As Δt becomes smaller and the number of time intervals becomes increasingly large, this time expansion becomes intractable.

Intelligent algorithms have been developed to deal with such problems [9]–[12]. Most of these techniques, however, use the assumption that the network has the FIFO property. This property states that if path 1 enters link L with a smaller total cost compared to the total cost of path 2, then path 1 also leaves link L with a smaller total cost. This assumption is reasonable if the traveling cost is the same as the traveling time (hence the name FIFO). As mentioned above, this is not generally the case. An example of a non-FIFO problem is that of searching for the safest route.

The goal of the EP algorithm is to estimate an upper bound and a lower bound on the total traveling cost of going from the source to the destination through all possible paths before the time expansion process (the estimation step), and then eliminate the paths that are unlikely to have the minimum cost (the pruning step). The output of the EP algorithm is a small number of candidate paths, which can be expanded in time in a tractable manner.

III. Basic Ideas of the Estimation (Forward) Step

We will define some new parameters as follows:

$Cf_{min}(i)$: The lower bound on the cost of traveling from source to node i

$Cf_{max}(i)$: The upper bound on the cost of traveling from source to node i

$T_{min}(i)$: The lower bound on the time of arrival at node i

$T_{max}(i)$: The upper bound on the time of arrival at node i .

Our goal in this step is to find the above parameters for all the nodes until we reach the destination node. Again, consider the simple example of Fig. 1. Assume that we already know $Cf_{min}(A)$ and $Cf_{max}(A)$. We also know that $T_{min}(A) = t_1$ and $T_{max}(A) = t_2 = t_1 + 3\Delta t$. We can now estimate the same parameters for node B as follows:

$$\begin{aligned} Cf_{min}(B) &= Cf_{min}(A) + \min\{C_{AB}(t_1^*) \mid T_{min}(A) < t_1^* < T_{max}(A)\} \\ Cf_{max}(B) &= Cf_{max}(A) + \max\{C_{AB}(t_2^*) \mid T_{min}(A) < t_2^* < T_{max}(A)\} \\ Tf_{min}(B) &= T_{min}(A) + \min\{T_{AB}(t_3^*) \mid T_{min}(A) < t_3^* < T_{max}(A)\} \\ Tf_{max}(B) &= T_{max}(A) + \max\{T_{AB}(t_4^*) \mid T_{min}(A) < t_4^* < T_{max}(A)\} \end{aligned} \quad (1)$$

These bounds on the costs and the arrival times may never be achieved, but they are absolute bounds and are never exceeded. This example suggests that, after initialization at the source node, one can proceed node to node toward the destination and calculate the cost and arrival time bounds on each node. If more than one branch enter a particular node, the minimum of the lower bounds and the maximum of the upper bounds are recorded (the latter is due to the non-FIFO property). This process continues until the destination node is reached. At this point, we can compute the bounds on the cost for entering the destination node through the last link. If k links enter the destination node at the end, we will have K bounds on the cost and arrival time for each link. We are now ready to traverse the network backward and prune the possible paths that have unpromising bounds on their traveling cost.

IV. Basic Ideas of the Pruning (Backward) Step

So far, in the forward step, we have been keeping the minimum of the lower bounds and the maximum of the upper bounds. In the backward step, we will keep the minimum of the lower bounds and the *minimum* of the upper bounds.

We shall define:

$Cb_{min}(i)$: The lower bound on the cost of traveling from node i to the destination.

$Cb_{max}(i)$: The upper bound on the cost of traveling from node i to the destination

These parameters can be updated from node to node in a similar way as in the forward step. Consider the situation in Fig. 3, where the destination node D can be reached from any of the K nodes.

For each node entering the destination node, we can calculate:

$$\begin{aligned} C_{min}^i &= Cf_{min}(i) + Cb_{min}(i) \\ C_{max}^i &= Cf_{max}(i) + Cb_{max}(i) \end{aligned}$$

Where, C_{min}^i is the lower bound on the cost of going from source to destination through node i , and C_{max}^i is the upper bound on the cost of going from source to destination through node i .

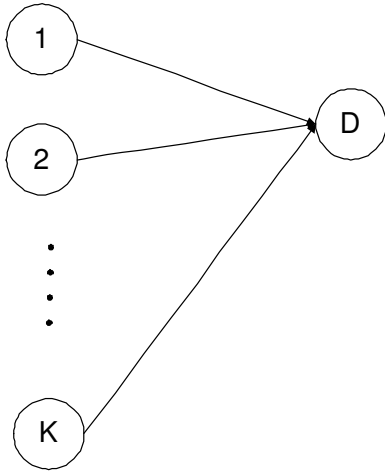


Fig. 3. K nodes entering destination

Fig. 4 shows a possible outcome of what these bounds may look like for $K = 5$ nodes entering the destination.

The function of the pruning step should now be clear. Compare the bounds of node 1 and node 4. Since $C_{max}^1 < C_{min}^4$, it is always better to get to node D through node 1 as opposed to node 4. This is the best case when the intervals within the bounds of different nodes do not overlap. In the basic EP algorithm presented in the next section, the pruning step always chooses at most two nodes in each stage. These two nodes are the nodes corresponding to the smallest minimum and the smallest maximum (which may happen to correspond to the same node). All other nodes are discarded. In the example of Fig. 4, only nodes 1 and 2 are selected, and this ensures that the final route will have a cost bounded by C_{min}^1 and C_{max}^2 . Section VII extends this to a more generic version of the EP algorithm that allows selecting the best $2p$ nodes at each pruning stage. By default, $p = 1$.

V. The EP Algorithm

A. Definitions

$\mathcal{V}$: List of all the nodes in the network (provided)

$\mathcal{Q}$: List of all the nodes to be processed by the forward step

S : Source node

D : Destination node

$T_{min}(i)$: Lower bound on the time of arrival at node i

$T_{max}(i)$: Upper bound on the time of arrival at node i

$Cf_{min}(i)$: Lower bound on travel cost from S to node i

$Cf_{max}(i)$: Upper bound on travel cost from S to node i

$Cb_{min}(i)$: Lower bound on travel cost from node i to D

$Cb_{max}(i)$: Upper bound on travel cost from node i to D

C_{min}^i : Lower bound on cost from S to D through node i

C_{max}^i : Upper bound on cost from S to D through node i

$C_{ij}(t)$: Travel cost from node i to node j at time t

$T_{ij}(t)$: Travel time from node i to node j at time t

$\mathcal{A}(i)$: List of nodes that are directly linked to from node i

$\mathcal{F}(i)$: List of nodes entering i collected in the forward step

$\mathcal{B}(i)$: List of nodes from $\mathcal{A}(i)$ kept in the backward step

B. Initialization

We initialize some of the above variables as follows:

$\mathcal{Q} = \mathcal{V}$

foreach i **in** $\mathcal{V}$ **do**

$T_{min}(i) = \infty$

$T_{max}(i) = 0$

$Cf_{min}(i) = \infty$

$Cf_{max}(i) = 0$

$Cb_{min}(i) = \infty$

$Cb_{max}(i) = \infty$

end

$T_{min}(S) = t_1$ (entered by user)

$T_{max}(S) = t_2$ (entered by user)

$Cf_{min}(S) = 0$

$Cf_{max}(S) = 0$

$Cb_{min}(D) = 0$

$Cb_{max}(D) = 0$

$\mathcal{F}(i) = NULL$

$\mathcal{B}(i) = NULL$

C. Forward (Estimation) Step

while D is in $\mathcal{Q}$ **do**

i = node with the smallest $Cf_{min}(i)$ from $\mathcal{Q}$

remove i from $\mathcal{Q}$

if $i = D$ **then**

break

end

foreach node j in intersection of $\mathcal{A}(i)$ and $\mathcal{Q}$ **do**

add node j to $\mathcal{F}(j)$

if $T_{min}(j) > T_{min}(i) + \min\{T_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$ **then**

$T_{min}(j) = T_{min}(i) + \min\{T_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$

end

if $T_{max}(j) < T_{max}(i) + \max\{T_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$ **then**

$T_{max}(j) = T_{max}(i) + \max\{T_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$

end

if $Cf_{min}(j) > Cf_{min}(i) + \min\{C_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$ **then**

$Cf_{min}(j) = Cf_{min}(i) + \min\{C_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$

end

if $Cf_{max}(j) < Cf_{max}(i) + \max\{C_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$ **then**

$Cf_{max}(j) = Cf_{max}(i) + \max\{C_{ij}(t^*) \mid$

$T_{min}(i) < t^* < T_{max}(i)\}$

end

end

end

D. Backward (Pruning) Step

```

 $\mathcal{L}_{current} = NULL$ 
 $\mathcal{L}_{next} = D$ 
while  $\mathcal{L}_{next} \neq NULL$  do
     $\mathcal{L}_{current} = \mathcal{L}_{next}$ 
     $\mathcal{L}_{next} = NULL$ 
    foreach node  $j$  in  $\mathcal{L}_{current}$  do
         $C_{min}^j = \min_{i \in \mathcal{F}(j)} \{Cf_{min}(i) + \min\{C_{ij}(t^*) \mid T_{min}(i) < t^* < T_{max}(i)\} + Cb_{min}(j)\}$ 
         $MIN(j) = \arg \min_{i \in \mathcal{F}(j)} \{Cf_{min}(i) + \min\{C_{ij}(t^*) \mid T_{min}(i) < t^* < T_{max}(i)\} + Cb_{min}(j)\}$ 
         $C_{max}^j = \min_{i \in \mathcal{F}(j)} \{Cf_{max}(i) + \max\{C_{ij}(t^*) \mid T_{min}(i) < t^* < T_{max}(i)\} + Cb_{max}(j)\}$ 
         $MAX(j) = \arg \min_{i \in \mathcal{F}(j)} \{Cf_{max}(i) + \max\{C_{ij}(t^*) \mid T_{min}(i) < t^* < T_{max}(i)\} + Cb_{max}(j)\}$ 
    end
     $n = \arg \min_{j \in \mathcal{L}_{current}} \{C_{min}^j\}$ 
     $m = MIN(n)$ 
    if  $Cb_{min}(m) > Cb_{min}(n) + \min\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$  then
         $Cb_{min}(m) = Cb_{min}(n) + \min\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$ 
    end
    if  $Cb_{max}(m) > Cb_{max}(n) + \max\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$  then
         $Cb_{max}(m) = Cb_{max}(n) + \max\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$ 
    end
    add  $n$  to  $\mathcal{B}(m)$ 
    if  $m \neq S$  then
        add  $m$  to  $\mathcal{L}_{next}$ 
    end
     $n = \arg \min_{j \in \mathcal{L}_{current}} \{C_{max}^j\}$ 
     $m = MAX(n)$ 
    if  $Cb_{min}(m) > Cb_{min}(n) + \min\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$  then
         $Cb_{min}(m) = Cb_{min}(n) + \min\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$ 
    end
    if  $Cb_{max}(m) > Cb_{max}(n) + \max\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$  then
         $Cb_{max}(m) = Cb_{max}(n) + \max\{C_{mn}(t^*) \mid T_{min}(m) < t^* < T_{max}(m)\}$ 
    end
    if  $n$  not already in  $\mathcal{B}(m)$  then
        if  $m \neq S$  AND  $m$  is not already in  $\mathcal{L}_{next}$  then
            add  $m$  to  $\mathcal{L}_{next}$ 
        end
    end
end
    
```

E. Route Optimization

The best route in the pruned network can now be computed using the following recursive algorithm:

```

function Route Optimization
     $opt\_route = NULL$ 
     $opt\_cost = \infty$ 
    call Propagate( $S, NULL$ )
    output  $opt\_route$  and  $opt\_cost$ 

function Propagate( $node, path$ )
     $path = path + node$ 
    if  $node = D$  then
        foreach  $Tmin(S) \leq t \leq Tmax(S)$  do
            find the best cost path
        end
        if lower cost is achieved then
            update  $opt\_route$  and  $opt\_cost$ 
        end
        return
    end
    else
        foreach  $n$  in  $\mathcal{B}(node)$  do
            call Propagate( $n, path$ )
        end
    end
    
```

VI. Complexity Analysis

The forward step in the EP algorithm has the same complexity as the Dijkstra algorithm, $O(N^2)$ [13]. The number of pruning iterations is at least equal to the number of nodes on the best path from the source to the destination. In the worst case, when the number of nodes in $\mathcal{F}$ linearly depends on N (as opposed to a constant), the pruning step could take at most $O(N^2)$. However, in typical networks the pruning step converges a lot faster than $O(N^2)$.

VII. Extension to the Algorithm

The basic EP algorithm presented in the previous section may eliminate the most optimal route in the pruning step. An extension of the EP algorithm could take a parameter p as the input, and select the best $2p$ nodes in each stage of the backward step (the factor of 2 is for the min and max criteria). This parameter is a tuning parameter that introduces a trade-off between the running time and the performance of the algorithm.

VIII. Simulation Results

The EP algorithm discussed in the previous sections has been implemented in Java for the purpose of concept testing and comparison with other methods. The main performance benchmark in the following simulation results is the Dijkstra algorithm adopted to compute optimal costs for static approximations of a dynamic network. The first approximation uses minimum cost across all the time bins as the link cost and we refer to this implementation as MIN_{DA} . The second approximation uses the average cost across all the time bins as the link cost and we refer to it as AVG_{DA} . After running MIN_{DA} and AVG_{DA} , the actual dynamic cost of their output route is calculated by traversing the dynamic network along these routes. In addition, the absolute minimum cost route for the given dynamic network is computed by exhaustive search.

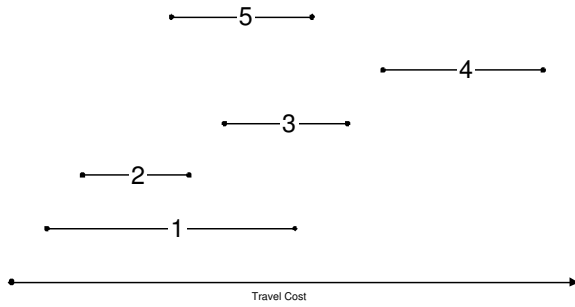


Fig. 4. Cost bound comparison for different paths

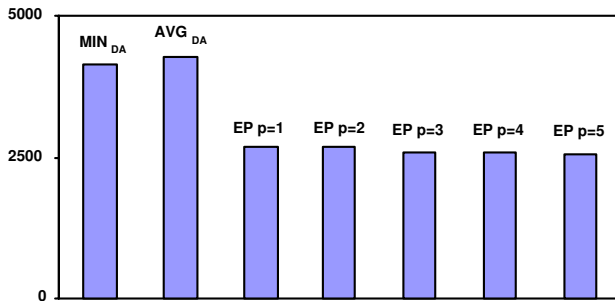


Fig. 5. Route Cost

All algorithms were tested and compared on randomly generated networks with different setups. The network variables are: number of nodes, number of links leaving each node, average travel time and cost on each link, standard deviation for time and cost of each link, number of time bins, and the neighborhood size. The neighborhood parameter was introduced to model the node clusters in the real road networks (e.g., an urban area is heavily connected within itself, but different areas are sparsely connected between each other). Higher time and cost values were also introduced for certain time bins in order to simulate time characteristics in the real world (e.g., rush hour conditions on the highways).

A numerical comparison was performed on a sample network with 1000 nodes, 3 links leaving each node, 100 node neighborhood, 24 time slots (hourly model), and uniformly distributed times and costs with some additional rush hour penalties at specified time bins. Obtained results are presented in Table I and Fig. 5.

The EP algorithm achieves significantly better results than the Dijkstra algorithms. For realistic road network of this simulation with rush hour traffic patterns, EP outperforms both Dijkstra approximations by 35-40%.

The EP routing cost in Table I is very close to the exhaustive search optimal routing cost even with pruning parameter $p = 1$ and as p is increased the EP cost rapidly converges to the optimal cost point.

IX. Conclusion

In this paper we have considered the problem of finding the best point-to-point route with minimum cost through a non-FIFO

TABLE I
SAMPLE RESULTS

Algorithm	Achieved Cost
MIN_{DA}	4135.58
AVG_{DA}	4290.27
EP ($p = 1$)	2680.96
EP ($p = 2$)	2680.96
EP ($p = 3$)	2597.58
EP ($p = 4$)	2597.58
EP ($p = 5$)	2559.41
EP ($p = 6$)	2559.41
EP ($p = 7$)	2559.41
Exhaustive Search	2559.41

dynamic network. We introduced a novel algorithm (EP) that estimates the lower and upper bounds on all the paths through the network and uses these bounds to prune the paths with higher costs. The algorithm then finds the optimal route for the pruned subset of network paths.

The basic ideas of the EP algorithm are presented together with pseudo-code. The experimental results verified the value of this new algorithm. The future directions of research include performance evaluation of the EP algorithm on dynamic networks of various nature as well as investigation of the real-time and distributed implementations.

References

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] N. Christofides, "Vehicle routing," in *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*, E. Lawler, J. Lenstra, A. R. Kan, and D. Shmoys, Eds. New York: John Wiley and Sons, 1985.
- [3] W. H. Randolph. (2002, Feb.) Vehicle Routing Software Survey, paper and online edition of OR/MS Today. [Online]. Available: http://www.lionhrtpub.com/orms/surveys/Vehicle_Routing/vrss.html
- [4] University of Maryland, College Park, MD. (2003) The Dynasmart website. [Online]. Available: <http://www.dynasmart.com/>
- [5] Massachusetts Institute of Technology, Cambridge, MA. (2003) The DynaMIT website. [Online]. Available: <http://mit.edu/its/>
- [6] M. Gendreau and J.-Y. Potvin, "Dynamic vehicle routing and dispatching," in *Fleet Management and Logistics*, T. Crainic and G. Laporte, Eds. Kluwer, 1998, pp. 115–126.
- [7] H. N. Psaraftis, "Dynamic vehicle routing: Status and prospects," *Annals of Operations Research*, vol. 61, pp. 143–164, 1995.
- [8] W. Powell, P. Jaillet, and A. Odoni, "Stochastic and dynamic networks and routing," in *Handbooks in Operations Research and Management Science, Network Routing*, M. Ball, T. Magnanti, C. Monma, and G. Nemhauser, Eds. Amsterdam: Elsevier, 1995, vol. 8, pp. 141–296.
- [9] I. Chabini and S. Lan, "Adaptations of the A* algorithm for the computation of fastest paths in deterministic discrete-time dynamic networks," *IEEE Trans. Intell. Transport. Syst.*, vol. 3, pp. 60–74, Mar. 2002.
- [10] I. Chabini and S. Gao, "Optimal routing policy problems in stochastic time-dependent networks. I. Framework and taxonomy," in *Proc. The IEEE 5th International Conference on Intelligent Transportation Systems*, 2002, pp. 549–554.
- [11] —, "Optimal routing policy problems in stochastic time-dependent networks. II. Exact and approximation algorithms," in *Proc. The IEEE 5th International Conference on Intelligent Transportation Systems*, 2002, pp. 555–559.
- [12] D. J. Bertsimas and D. Simchi-Levi, "A new generation of vehicle routing research: Robust algorithms, addressing uncertainty," *Operations Research*, vol. 44, pp. 286–304, 1996.
- [13] T. H. Cormen (Editor), C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 2nd ed. MIT Press, Sept. 2001.